

Chetan Babu Mahendiran

Sarcasm Detection with ML

For the module COMP9060 as part of the

Master of Science in Data Science and Analytics, Department of Mathematics, 2025/26

Table of Contents

Sarcasm Detection with ML *3*

Overview *3*

Task 1 - Tokenisation & Preprocessing *3*

Task 2 - Manual Feature Representation *4*

Task 3 - Multinomial & Bernoulli Naïve Bayes *5*

Task 4 - Numeric Features & Imputation *6*

Task 5 - Outlier Detection *8*

Task 6 - Scaling *8*

Task 7 - Class Imbalance *9*

Task 8 - Error Analysis *10*

Sarcasm Detection with ML

Overview

This model is built for the News Headlines Dataset from Kaggle, the aim is to build a model to detect sarcasm from the New Headlines using a preprocessing pipeline and fit Naïve Bayes Model.

Seed: 77227

Smoothing set: [0, 0.5, 2]

Ablation index: 3

Task 1 - Tokenisation & Preprocessing

Basic Cleaning:

- Lowercasing: All text is converted to lowercase so that 'Trump' and 'trump' are treated as the same word.
- HTML tag removal: Using re.sub(r'<.*?>', '', text) to strip any leftover HTML markup from the raw data.
- Sarcasm marker substitution: Before removing punctuation, specific tonal markers are replaced with unique tokens - '!!' becomes [EXC_SEQ], '??' becomes [QUE_SEQ], combinations of '?!' or '!?' become [INT_MARK], and '...' becomes [ELLIP]. These markers are important because they often signal sarcasm.
- Punctuation removal: All remaining punctuation is removed except apostrophes inside words (e.g., "don't" is preserved).
- Tokenisation: Final tokens are extracted using regex: r"\b\w+'\w+|\w+\b|[EXC_SEQ]|[QUE_SEQ]|[INT_MARK]|[ELLIP]"

Negation Handling:

Negation changes the meaning of words that follow it. For example, "not good" should be treated differently from "good". The implementation counts the number of vowels in each negation word and uses that count (N) to prefix the next N tokens with NOT_

Stop word Removal:

A custom stop word list of 43 words was defined, covering common articles, conjunctions, pronouns, and prepositions (e.g., a, an, the, and, or, but, in, on, at, for, with, about, to, from, by, of, as). Stop words are removed only if the token does not start with NOT_. This ensures that negated stop words like NOT_a are preserved, as they carry semantic meaning in context.

N – Gram Generation:

Both unigrams (individual tokens) and bigrams (consecutive token pairs) are generated for each headline. Bigrams are joined with an underscore, e.g., 'man_arrested'. This captures short phrases and improves the model's ability to detect sarcastic patterns that span two words.

OOV Handling:

The vocabulary is built exclusively from the training set. Any token in the validation or test set that does not appear in the training vocabulary is replaced with the special token <UNK>. This prevents unseen words from causing error.

Train - Validation – Test :
Dataset has been split into with Train (70%), Validation (15%) and Test set(15%)

```
Train size: 20033
Validation size: 4293
Test size: 4293
Example validation features: ['harvey', '<UNK>', 'tornadoes', '<UNK>', 'homes', 'outside', 'houston', '<UNK>', '<UNK>', '<UNK>',
'<UNK>', '<UNK>', '<UNK>']
```

Task 2 - Manual Feature Representation

Count Vectorizer:

The count vectorizer was built entirely from scratch using the training data:

- Document Frequency (DF) was calculated by counting how many training documents each token appeared in (counting each token at most once per document).
- Rare word filtering: Tokens appearing in fewer than t = (77227 mod 6) + 2 = (1) + 2 = 3 documents were removed.
- The remaining tokens were sorted in strict alphabetical order.
- Tokens at prime-numbered positions (2, 3, 5, 7, 11 ...) were assigned their own feature column. Tokens at non-prime positions were aggregated into a single 'Non-Prime' feature at index 0.
- The result is a document-term matrix where each row represents a headline and each column is either a prime-indexed token or the non-prime aggregate.

TF-IDF:

TF (Term Frequency) was computed as the count of a term in a document divided by the total number of terms in that document. IDF (Inverse Document Frequency) was computed as log(N / df) where N is the number of training documents and df is the document frequency of the token. The final TF-IDF score is the element-wise product of TF and IDF, giving higher weight to terms that are distinctive to specific documents.

Top 10 Tokens by Document Frequency (Training Set):

```
Top 10 tokens by Document Frequency:
new: 1142
man: 980
trump: 949
after: 685
out: 645
up: 589
how: 583
not: 565
just: 468
his: 467
```

Task 3 - Multinomial & Bernoulli Naïve Bayes

Multinomial:

Multinomial NB estimates the probability of a document belonging to a class based on word frequencies. All computations are performed in log-space to avoid numerical underflow.

- Priors: P(sarcastic) and P(not sarcastic) are estimated from the training class distribution.
- Likelihoods: P(word | class) = (count(word, class) + α) / (total_words_in_class + α × V), where V is the vocabulary size.
- Laplace smoothing: Three alpha values from the seed-determined set {0.1, 0.5, 1} were tested on the validation set. The best-performing alpha was selected.

Bernoulli:

Bernoulli NB models the presence or absence of each word in a document as a binary feature. For each token in the vocabulary, the model estimates P(word present | class) and P(word absent | class). During prediction, both present and absent words contribute to the final score — this is the key difference from Multinomial NB.

Bernoulli vs Multinomial for Sarcasm Detection

Sarcasm in news headlines often depends on specific word choice rather than how many times a word appears. A headline like 'Trump solves world hunger' is sarcastic because of which words appear, not because any word is repeated. This makes Bernoulli NB a conceptually strong fit for this task.

Multinomial NB rewards word frequency, which typically matters more in longer documents like articles or reviews. For short headlines of 7–12 words on average, the frequency signal might seem weak - however, the inclusion of bigrams in the feature set effectively extends document length and creates richer frequency patterns that Multinomial NB can leverage.

In practice, Multinomial NB significantly outperformed Bernoulli NB on the validation set, achieving 81% accuracy compared to 66% for Bernoulli. This result can be explained by the fact that the preprocessing pipeline includes bigrams, which adds frequency signal that Multinomial NB exploits effectively. The vocabulary is also large due to bigram expansion, meaning word frequency patterns across sarcastic and non-sarcastic classes become more distinguishable. Bernoulli NB's lower performance suggests that simply knowing whether a word appears or not is not sufficient — how often certain phrases and bigrams occur within each class provides meaningful additional signal. For this specific dataset, Multinomial NB is the stronger choice.

Task 4 - Numeric Features & Imputation

Feature	Description
Total_Word_Count	Number of words in the raw headline
Char_Count	Total number of characters
Uppercase_Count	Number of fully uppercase words (length > 1)
Uppercase_Ratio	Uppercase_Count / Total_Word_Count

Exclamation_Count	Number of '!' characters
Exclamation_Ratio	Exclamation_Count / Total_Word_Count
Question_Count	Number of '?' characters
Question_Ratio	Question_Count / Total_Word_Count
Negation_Count	Count of negation words in headline
Digit_Count	Number of digit characters
Digit_Ratio	Digit_Count / Total_Word_Count
Avg_Word_Length	Char_Count / Total_Word_Count
Sarcasm_Ratio	Uppercase_Count / Total_Word_Count (0.5 if word count is 0)

Missing Value Injection:

Using the seed (77227), 5% of values in each numeric column were randomly set to NaN. An additional check ensures at least one NaN appears in the Sarcasm_Ratio column, as required. This simulates real-world missing data scenarios.

Missing Value Imputation:

Two imputation strategies were implemented:

- Mean Imputation: Each missing value is replaced with the overall column mean, calculated from all non-missing values in that column.
- Class-Conditional Mean Imputation: Missing values are replaced with the mean for that feature within the same class (sarcastic vs. not sarcastic). This is computed using groupby on is_sarcastic.

Comparison:

Class-conditional mean imputation is the better approach for this task. Sarcastic and non-sarcastic headlines have genuinely different feature distributions. For example, sarcastic headlines tend to use more uppercase words and exclamation marks. Using overall mean imputation blends these distributions together, inserting values that do not accurately reflect

the class a headline belongs to. This introduces noise into the data and weakens the model's ability to distinguish between the two classes. Class-conditional imputation preserves the statistical differences between classes, leading to more informative feature values after imputation and better downstream model performance.

Task 5 - Outlier Detection

IQR-Based Outlier Detection

For each numeric feature, the IQR (Q3 - Q1) was calculated. The lower bound is Q1 - 1.5×IQR and the upper bound is Q3 + 1.5×IQR. Values outside these bounds are classified as outliers. Features like Exclamation_Count and Digit_Count had IQR = 0 because most headlines contain none of these characters, meaning any non-zero value is technically flagged as an outlier. Total_Word_Count showed the most meaningful spread.

Outlier Treatment: Clamping

Clamping was chosen over row removal because it retains all data while limiting the influence of extreme values. Every feature value above the upper bound is set to the upper bound, and every value below the lower bound is set to the lower bound. Only features with IQR > 0 were clamped.

Effect on Performance and Class Distribution

Clamping did not change the class distribution - all rows were retained, with approximately 14,985 non-sarcastic and 13,634 sarcastic headlines remaining after treatment. The model's performance was not significantly affected because most features had IQR = 0, meaning clamping only acted on a small subset of feature values. However, clamping Total_Word_Count to the IQR boundary prevents unusually long headlines from disproportionately influencing the Gaussian NB model. The main benefit is increased numerical stability and reduced sensitivity to extreme edge cases in the data.

Task 6 - Scaling

Both scalers were fit on the training data only and then applied to validation and test sets - this prevents data leakage:

- MinMax Scaling: Scales each feature to the range [0, 1] by computing (x - min) / (max - min). Preserves relative distances between values.
- Standardisation (Z-score): Scales each feature to have mean = 0 and standard deviation = 1, computed as (x - mean) / std.

Validation Accuracy with MinMax Scaling: 0.4998835313300722

Validation Accuracy with Standardisation: 0.4998835313300722

Comparison:

Both MinMax Scaling and Standardisation produced identical validation accuracy (~0.50), showing that scaling had little to no impact on Gaussian Naïve Bayes performance for this dataset. Although Standardisation typically benefits Gaussian NB by centring features around zero with unit variance to better approximate a normal distribution, many numeric features here are heavily skewed or zero-heavy (e.g., Exclamation_Count, Digit_Count). This likely prevented any improvement. MinMax Scaling, which compresses all values into [0, 1], similarly did not help because it distorts the Gaussian assumption when features are sparse or unevenly distributed. The validation accuracy being close to 0.50 indicates that the model is performing roughly at random, suggesting that the numeric features alone are weak predictors for sarcasm in headlines and that Gaussian NB may not be well-suited to this dataset.

Task 7 - Class Imbalance

Measuring Class Imbalance:

The training set contains approximately 14,985 non-sarcastic headlines (class 0) and 13,634 sarcastic headlines (class 1), giving a ratio of roughly 52:48. While this is a mild imbalance, it is enough to bias the model toward the majority class if left unaddressed.

Random Oversampling:

Random oversampling was applied to the minority class (sarcastic headlines) using sklearn.utils.resample with the student seed. The minority class was sampled with replacement until it matched the majority class size. The combined dataset was then shuffled. After oversampling, both classes have equal representation in the training data.

Threshold Tuning & Evaluation Metrics:

The model was evaluated at thresholds 0.3, 0.4, 0.5, 0.6, and 0.7. Results at threshold = 0.5:

```
Confusion Matrix (TP, FP, FN, TN):
TP: 2031, FP: 2123, FN: 24, TN: 115
Precision: 0.4889
Recall: 0.9883
F1 Score (Macro): 0.3755
F1 Score (Minority Class): 0.6542
Accuracy: 0.4999
```

Class Imbalance effects on Performance:

When class imbalance is present, a model can achieve deceptively high accuracy simply by predicting the majority class for every input. In this dataset, a model that always predicts 'not sarcastic' would be correct about 52% of the time - which is only slightly worse than a random classifier. After applying random oversampling, the model was forced to learn patterns from both classes equally, which improved recall for the sarcastic (minority) class. However, oversampling also introduced repeated examples, which can cause the model to overfit to specific sarcastic headlines rather than learning general patterns. The F1 score provides a more honest picture than accuracy alone, as it balances precision and recall and is more sensitive to performance on the minority class.

Task 8 - Error Analysis

Misclassified Headlines from Test Set

Eight misclassified headlines were examined from the test set predictions. These were identified by comparing the model's predictions against the true labels. Common patterns in the errors include:

- Subtle sarcasm missed: Sarcastic headlines that use no uppercase words, no exclamation marks, and no question marks look identical to serious headlines from a numeric feature perspective. The model only sees surface statistics, not meaning.
- Serious headlines flagged: Non-sarcastic headlines containing numbers, acronyms, or capitalised names can push numeric features like Digit_Count or Uppercase_Count above average, causing the model to incorrectly classify them as sarcastic.
- Short headlines: Very short headlines have high variance in ratios like Sarcasm_Ratio and Uppercase_Ratio because the denominator (Total_Word_Count) is small — a single uppercase word in a 3-word headline produces a ratio of 0.33, which looks unusual.
- Negation context: Headlines with negation words that are genuinely serious (e.g., 'police did not find evidence') can be misclassified because Negation_Count is elevated.

The overall error pattern confirms that numeric surface features are insufficient for reliable sarcasm detection.

New Head Lines:

```
Headline: Trump and Iran got compromised
Predicted Label: 1 (Prob=0.96)

Headline: USA Dollar has crashed
Predicted Label: 1 (Prob=0.97)

Headline: Assigngments are completed, Should i worry about the job?
Predicted Label: 0 (Prob=0.00)
```

For the three new headlines, the model predicted “Trump and Iran got compromised” as sarcastic with a high probability (0.96), and “USA Dollar has crashed” also as sarcastic with a high probability (0.97), while “Assignments are completed, should I worry about the job?” was predicted as not sarcastic with a probability of 0.00. These results indicate that the model is strongly biased toward predicting the sarcastic class, even for headlines that are clearly serious. While the first prediction aligns with human interpretation, the second is a clear misclassification, showing that the model struggles to distinguish genuine news from sarcasm. The third headline, which contains a question and could carry a sarcastic tone, was incorrectly classified as non-sarcastic, further highlighting inconsistency. This behavior reflects a key limitation of numeric-only feature models: they rely on surface-level patterns such as word count or punctuation and lack the ability to understand context, tone, or semantic meaning. As a result, the model tends to overpredict sarcasm and fails to generalize well to unseen examples.